

Micro Task Floods

مشاكل عنها اي حاجة ما شاء الله

التي فهمت من ChatGPT المصدر الوحيد
له الى انكلمني عنها:

ان ده بسبب ان في كذا microTask بتحصل بعشائر كبيرة
جدا وده هيتيج عنك block event loop
الـ JavaScript هيسلم في تنفيذ كل الـ microTask قبل ما ينفذ الـ macroTask
الافضل لو مفطر انك هتعمل حاجة زي كده مثلا في
Promise بتعملها 1000 ping [2000] مرة الافضل انك توزعها
شوية على الـ microTask و الافضل انك تخلصهم كمان جوة
SetTimeout مثلا

```
Function Step(timestamp) {
  div1.style.transform = "translate X(" + pos1
+ " px)";
```

المفروض كده برضو بكتب في ال CSS

```
pos1++;
if (pos1 <= 200) {
```

```
  requestAnimationFrame(step)
}
```

Function ← Parameter
 requestAnimationFrame (step)

طبعا هنا أنا بكتب callback لان بحتاج ال

requestAnimationFrame
 هذا في طبعا الافضل هو الى فوق في الصورة وهو

هنا
 Animation

requestFrame
 طبعا ال
 وده بيقلول انها تبدأ أول حاجة في Frame الشاشة حتى لو
 أنا كاتب Set Interval الاول

بمعنى كم مرة الشاشة تعرض صورة جديدة في الثانية الواحدة
 حلها كان بدل ما استخدم SetInterval استخدم requestAnimationFrame
 كل اما الشاشة تعمل refresh وتعمل Callback لا
 window بحيث انك لما تحصل للـ window انك يتغير الـ Frame
 لى بعد كده تنفذ الـ Function

هنتخذ المثال ده :
 let pos1 = 0
 let pos2 = 0
 let div1 = document.getElementById("div1")
 let div2 = document.getElementById("div2")

لو عايناه دول الفكرة بتاعتهم انا عاوز اوضح الفرق بين
 SetInterval و requestAnimationFrame
 let movement = setInterval(function() {
 div2.style.transform = "translateX(" +
 pos2 + "px)";
 pos2++;
 if (pos2 > 200) {
 clearInterval(movement);
 }
 } 16.666

طبعاً بتربطها على الرقم الى على الـ Frame الواحد
 طبعا هنا المشكلة في ان الـ SetInterval مش مالتيا مع الـ Frame الشاشة
 حتى لو تربطها مع 16.667 الوقت الى يتجدد فيه الـ Frame هنلاقي
 فيه زفقه كده آخى الـ requestAnimationFrame

Frame

هنفهم هنا ال KMP Script Channel

بص يا معلم ال Frame ده اشبه بشكل ال video ال Browser يرسم الشاشة 60 مرة في الثانية

$$1000ms/60 = 16.76$$

ده معناه ان كل Frame بيتزل كل 16ms
وال Frame ده عملياً كالاتي

1
2
3

مثلاً
مش الافضل خالص هنكتشف ده
افضل حاجة وانت بتعمل SetInterval على ال Frame انك بتصدر وفتح نفس
توقيت كل Frame يعني 16.6667
عشرات يطلع Smooth

المفروض ان ال requestAnimationFrame
Animation
بتأخذ بتحل مشكلة انك مش عارف ال refresh rate بتاعت ال user
هو عدد المرات الى ال browser جازاك بتحدث نفسك كل ثانية

long Task:

وبول أكثر، long Task ويمكن تسويتها بنفس وعرفت د لوقت
أكبر مثال على الـ long Task وهو الـ Portfolio عندنا أنا (٥) أنا عندي في
الـ background animation واحد جاهز من مكتبة ومستدعيه المشكلة
اني لما بعمل الـ scroll بيحصل تقطيع في الـ Animation علشان اكتشفت
د لوقت ان هذا المشكلة هتلاقيها ظهرت أكثر لما تقترح من الـ mobile

هو اي سايته هتأخذ وقت في تنفيذها أكثر من 50ms بدون تاثير فريسة
الـ Browser لما يعمل render علشان تكتشف ان الـ Browser
بيحاول يرسف Frame كل 16ms ← هفهمها لاي هو الـ Frame

المهم ان هيعمل بطء سبب الـ long Task وكمان هيعمل
تقطيع في الـ animation (ده نفسه الى حصل معايا) لما تعمل الـ scroll
تتجعد الـ UI
أكبر مثال على الـ long Task انك تعمل زي ما انما عملت كده (٥)
الـ setTimeout وتخليها بوقت كبير جداً او تعمل مثلاً الـ for loop او
الـ mapping وتخليها بوقت او تنفيذ كبير جداً وده طبعا بيمنع الـ event loop
انها تشتغل اي تاسك لحد ما تخلص الـ tasks الى على الـ micro Task
افضل حل للحوار هـ وهو الـ chunking
وده انا بقسم الشغل اللي بيأخذ وقت كبير جداً لـ أجزاء صغيرة
بتنفذ على مراحل علشان هيعمل اي تهيج

Day 6

call back

اولا كده تعالى نسين ال حاجات دول

1) micro Task:

احنا فهمنا ان ال Call Stack بيتخزن فيه كل حاجة جاية من ال Web API وبعد كده بنفكر ان ال Task queue بتخزن عليهم واحدة واحدة علشان ترتبهم من جوه ال micro Task يعني ترتبها الاول زي مثلا Promise دي بتنفذ الاول وليها اولوية اعلى من ال (macro) Task queue

2) macro Task:

دي بتنفذ ثاني حاجة ودها زي set Interval, set Timeout, I/O events, UI Event
لي دي من الاولوية علشان ال event loop هو اللي بيخلص عليهم واحدة واحدة

مثال

console.log('A')

setTimeout(() => console.log('B'), 5)

const myPromise = new Promise((resolved, rejected) => {
resolved('C')
})

myPromise.then((value) => console.log(value))

console.log('D')

النتيجة:

A, B

هنا في الاول علشان يدخلوا في ال web API
ها يدخلوا في ال Direct Call Stack وبعدين علشان هما sync و الاخر.

C

B

علشان ال micro Task تنفذ في الاول علشان لها الاولوية

علشان ال macro Task لها الاولوية الثانية في الاولوية